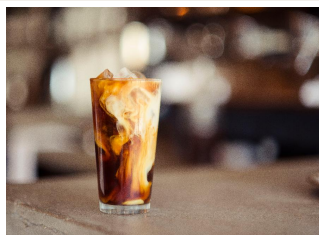


LUNACAFFE POINT OF SALES SYSTEM

Panduan UAS - Implementasi Pemrograman Berorientasi Objek

Studi kasus aplikasi kasir kafe berbasis Java Swing, SQLite, MVC, DAO, dan konsep OOP.



Komponen	Keterangan
Mata kuliah	Pemrograman Berorientasi Objek
Jenis proyek	Aplikasi desktop Point of Sales
Teknologi	Java Swing, FlatLaf, SQLite JDBC
Pola rancangan	MVC dan DAO
Tim	Firgiawan Listianto, Muh. Nabil Makarimsyah, Jumaria, Kelvin Surya Putra

DAFTAR MATERI

Peta Belajar PDF

PDF ini disusun sebagai bahan belajar dan pegangan presentasi UAS. Fokus utama dokumen adalah menjelaskan bagaimana konsep OOP diterapkan pada kode LunaCaffe POS, bukan hanya menjelaskan tampilan aplikasi.

Bab	Pokok Bahasan	Target Pemahaman
1	Gambaran sistem	Memahami ruang lingkup aplikasi dan aktor pengguna.
2	Arsitektur	Memahami pemisahan Model, View, DAO, dan Utility.
3	Implementasi OOP	Mampu menjelaskan class, object, encapsulation, inheritance, abstraction, dan polymorphism.
4	Database	Memahami struktur tabel dan hubungan data transaksi.
5	Alur kerja	Memahami perjalanan data dari katalog hingga pesanan selesai.
6	Panduan presentasi	Menyiapkan jawaban saat dosen menanyakan implementasi proyek.

BAB 1

Gambaran Umum Sistem

LunaCaffe POS adalah aplikasi kasir desktop yang mensimulasikan proses operasional kafe. Sistem dibagi menjadi antarmuka pelanggan dan dashboard pegawai. Pelanggan membuat pesanan melalui katalog, sedangkan kasir dan admin mengelola pesanan serta data operasional melalui panel internal.

Inti sistem

Aplikasi ini menunjukkan bahwa objek dalam dunia nyata seperti pelanggan, pegawai, menu, pesanan, dan detail pesanan dapat dimodelkan menjadi class Java yang saling berhubungan.

Fitur utama sistem:

- Katalog produk dengan gambar lokal, pencarian, filter kategori, dan pengurutan harga.
- Keranjang pelanggan dengan kontrol kuantitas dan validasi stok.
- Checkout yang menyimpan pesanan, detail pesanan, dan mengurangi stok secara transaksional.
- Dashboard pegawai untuk melihat antrean pesanan dan menandai pesanan selesai.
- Panel admin untuk tambah, ubah, hapus produk, restok, ubah harga, pilih gambar, dan kelola akun.
- Export rekapitulasi transaksi harian ke format CSV.

Aktor sistem:

Aktor	Cara Masuk	Hak Akses
Pelanggan	Tanpa login	Melihat katalog, mengatur keranjang, checkout, menerima struk.
Kasir	Login kasir1 / 123	Melihat pesanan live dan memperbarui status pesanan.
Admin	Login admin / 123	Mengelola produk, stok, harga, gambar, akun pegawai, dan laporan.

BAB 2

Arsitektur Program

Struktur program menggunakan pembagian tanggung jawab agar kode lebih mudah dibaca dan dikembangkan. Pola yang digunakan adalah MVC dengan tambahan DAO sebagai lapisan akses database.

Lapisan	Contoh File	Fungsi
Model	Menu, Pesanan, DetailPesanan, Aktor, Pegawai, Admin, Kasir, Pembeli	Merepresentasikan data dan perilaku inti dari domain aplikasi.
View	MainFrame, CustomerPanel, LoginPanel, DashboardPanel	Menyediakan antarmuka Java Swing untuk pengguna.
DAO	MenuDAO, OrderDAO, UserDAO	Menjalankan query SQL dan memisahkan akses database dari tampilan.
Utility	DatabaseConnection, DatabaseSeeder, SessionManager	Mengelola koneksi SQLite, data awal, dan sesi login.

Alur dependensi sederhana:

```
View -> DAO -> DatabaseConnection -> SQLite
View -> Model
DAO -> Model
SessionManager -> Pegawai
```

Penjelasan arsitektur:

- View tidak menyusun query SQL secara langsung. Ketika membutuhkan data menu atau pesanan, view memanggil DAO.
- DAO bertanggung jawab mengambil data dari SQLite dan mengubahnya menjadi object model.
- Model tidak mengetahui detail tampilan. Model hanya menyimpan data dan perilaku domain.
- Utility mendukung fungsi lintas modul seperti koneksi database dan sesi login.

BAB 3

Pemetaan Class dan Object

Bagian ini menunjukkan class utama yang menjadi fondasi aplikasi. Pemetaan ini penting untuk menjawab pertanyaan UAS tentang struktur objek dan relasi antar class.

Class	Peran	Konsep OOP yang terlihat
Aktor	Class abstrak untuk semua aktor sistem.	Abstraction
Pembeli	Turunan Aktor untuk pelanggan kiosk.	Inheritance, Polymorphism
Pegawai	Turunan Aktor untuk pengguna internal dengan username, password, dan role.	Inheritance, Encapsulation
Admin	Turunan Pegawai dengan kemampuan pengelolaan menu dan akun.	Inheritance
Kasir	Turunan Pegawai dengan kemampuan memperbarui status pesanan.	Inheritance
Menu	Model produk yang memiliki harga, stok, gambar, dan label.	Encapsulation
Pesanan	Model transaksi utama yang memiliki banyak DetailPesanan.	Object composition
DetailPesanan	Penghubung antara Pesanan dan Menu beserta qty dan subtotal.	Object composition

Relasi class inti:

```
Aktor
-> Pembeli
-> Pegawai
    -> Admin
    -> Kasir

Pesanan
-> List<DetailPesanan>
-> Menu
```

Catatan UAS
Jika diminta menjelaskan struktur abstrak class, mulai dari Aktor sebagai class abstrak, lanjutkan ke Pegawai dan Pembeli, kemudian turunan Pegawai yaitu Admin dan Kasir.

BAB 4

Implementasi Enam Konsep OOP

Bagian ini adalah inti pembahasan UAS. Setiap konsep dijelaskan berdasarkan kode nyata yang ada di project.

1. Class

Class adalah cetak biru untuk membentuk object. Pada project ini, contoh paling mudah adalah class Menu, Pesanan, dan Pegawai. Class Menu menyimpan identitas produk, kategori, harga, stok, path gambar, dan status label.

```
public class Menu {  
    private String idMenu;  
    private String namaMenu;  
    private String kategori;  
    private double harga;  
    private int stok;  
}
```

2. Object

Object adalah instance nyata dari class. Saat pelanggan memesan, sistem membentuk object Pesanan dan DetailPesanan. Ketika DAO membaca data dari database, DAO juga membentuk object Menu.

```
Menu m = new Menu(id, nama, kategori, harga, stok, imagePath, isNew, isBestseller);  
Pesanan p = new Pesanan("TEMP", "Guest", "");
```

3. Encapsulation

Encapsulation terlihat dari penggunaan atribut private pada model. Atribut seperti harga, stok, dan password tidak diakses langsung dari luar class, tetapi melalui method yang disediakan.

```
private double harga;  
private int stok;  
  
public double getHarga() { return harga; }  
public int getStok() { return stok; }  
public void setHarga(double harga) { this.harga = harga; }
```

BAB 4 LANJUTAN

Inheritance, Abstraction, dan Polymorphism

4. Inheritance

Inheritance digunakan untuk menurunkan karakteristik umum. Class Pegawai mewarisi Aktor. Class Admin dan Kasir mewarisi Pegawai. Dengan cara ini, atribut umum seperti id dan nama tidak perlu ditulis ulang di setiap class.

```
public class Pegawai extends Aktor { ... }
public class Admin extends Pegawai { ... }
public class Kasir extends Pegawai { ... }
```

5. Abstraction

Class Aktor dibuat abstract karena aktor umum tidak langsung digunakan sebagai objek konkret. Aktor hanya menjadi konsep umum yang mewajibkan turunan memiliki method masukSistem().

```
public abstract class Aktor {
    protected String id;
    protected String nama;

    public abstract void masukSistem();
}
```

6. Polymorphism

Polymorphism terlihat dari method masukSistem() yang didefinisikan di Aktor, kemudian diimplementasikan berbeda oleh Pembeli dan Pegawai. Nama method sama, tetapi perilaku class turunannya dapat berbeda.

```
@Override
public void masukSistem() {
    System.out.println("Pembeli " + nama + " memasuki sistem self-service.");
}

@Override
public void masukSistem() {
    System.out.println("Pegawai " + nama + " (" + role + ") memasuki sistem.");
}
```

Jawaban singkat saat presentasi

Polymorphism di project ini dapat dijelaskan melalui method masukSistem(). Method tersebut ada sebagai kontrak di Aktor, lalu Pembeli dan Pegawai memberikan implementasi sesuai peran masing-masing.

BAB 5

Object Composition pada Pesanan

Selain inheritance, project ini juga menunjukkan composition. Pesanan memiliki daftar DetailPesanan. Setiap DetailPesanan memiliki satu Menu. Pola ini cocok untuk transaksi karena satu pesanan dapat memiliki banyak produk.

```
public class Pesanan {
    private List<DetailPesanan> items;

    public void addItem(DetailPesanan detail) {
        this.items.add(detail);
        this.totalHarga = hitungTotal();
    }
}

public class DetailPesanan {
    private Menu menu;
    private int qty;
    private double subtotal;
}
```

Mengapa composition penting:

- Pesanan tidak menyimpan nama menu secara terpisah, tetapi menyimpan object DetailPesanan.
- DetailPesanan menyimpan referensi ke Menu sehingga harga dan identitas menu tetap terhubung.
- Subtotal dihitung berdasarkan harga menu dikalikan qty.
- Total pesanan dihitung dari seluruh subtotal item.

Rumus sederhana:

```
subtotal = menu.getHarga() * qty
totalHarga = sum(subtotal semua DetailPesanan)
```


BAB 6

DAO dan Operasi Database

DAO atau Data Access Object digunakan agar operasi database tidak bercampur dengan kode tampilan. View cukup memanggil method DAO, sedangkan detail SQL dikelola di class DAO.

Contoh MenuDAO

MenuDAO bertanggung jawab mengambil semua menu, menambah stok, mengubah harga, menambah menu, memperbarui menu, menghapus menu, dan membuat ID menu berikutnya.

Method	Fungsi
getAllMenus()	Mengambil seluruh data produk dari tabel menus.
addMenu(Menu menu)	Menambahkan produk baru ke database.
updateMenu(Menu menu)	Memperbarui nama, kategori, harga, stok, gambar, dan label produk.
deleteMenu(String idMenu)	Menghapus produk berdasarkan ID.
addStok(String idMenu, int qtyAdded)	Menambah stok produk.
updateHarga(String idMenu, double hargaBaru)	Mengubah harga produk.

Contoh OrderDAO

OrderDAO memproses checkout dengan transaksi database. Sistem memasukkan data order, memasukkan detail order, dan mengurangi stok. Jika stok tidak cukup, transaksi dibatalkan melalui rollback.

```
String sqlUpdateStok = "UPDATE menus SET stok = stok - ? WHERE id = ? AND stok >= ?";

conn.setAutoCommit(false);
...
conn.commit();
...
conn.rollback();
```

BAB 7

Database SQLite

Database SQLite digunakan karena sederhana, lokal, dan tidak membutuhkan server. File database bernama lunacaffe.db berada di root project. Saat aplikasi pertama kali berjalan, tabel dibuat otomatis melalui `DatabaseConnection.initializeDatabase()`.

Tabel	Fungsi	Kolom utama
users	Menyimpan akun pegawai.	id, nama, username, password, role
menus	Menyimpan produk menu.	id, nama, kategori, harga, stok, image_path, is_new, is_bestseller
orders	Menyimpan transaksi utama.	id, nama_pelanggan, total_harga, status, created_at
order_details	Menyimpan detail item transaksi.	order_id, menu_id, qty, subtotal

Relasi data:

```
orders.id -> order_details.order_id
menus.id  -> order_details.menu_id
```

Akses database

Untuk kebutuhan pengguna, admin dapat mengelola produk lewat panel aplikasi. Untuk pemeriksaan manual, file lunacaffe.db dapat dibuka dengan DB Browser for SQLite.

BAB 8

Alur Kerja Sistem

Alur kerja menjelaskan perjalanan data sejak pelanggan memilih menu sampai pesanan muncul di dashboard kasir.

Alur pelanggan

- CustomerPanel memuat data menu dari MenuDAO.getAllMenus().
- Pelanggan memilih menu, lalu sistem membuat DetailPesanan dan menambahkannya ke Pesanan.
- Saat qty berubah, DetailPesanan menghitung ulang subtotal.
- Pesanan menghitung total melalui hitungTotal().
- Saat checkout, OrderDAO.createOrder() menyimpan order dan detail order ke SQLite.
- Stok menu dikurangi menggunakan query update yang memeriksa ketersediaan stok.

Alur pegawai

- LoginPanel memanggil UserDAO.getUserByUsername().
- Pegawai diverifikasi melalui method verifikasiLogin().
- SessionManager menyimpan user aktif.
- DashboardPanel menampilkan menu sesuai role.
- Kasir dapat menandai pesanan selesai melalui OrderDAO.updateStatus().
- Admin dapat mengelola produk melalui MenuDAO dan mengelola akun melalui UserDAO.

```
Pelanggan -> CustomerPanel -> Pesanan -> OrderDAO -> SQLite  
Kasir/Admin -> LoginPanel -> UserDAO -> SessionManager -> DashboardPanel
```

BAB 9

Panduan Menjelaskan Saat UAS

Bagian ini berisi format jawaban singkat yang dapat digunakan saat presentasi atau tanya jawab.

Pertanyaan Dosen	Jawaban yang Disarankan
Apa aplikasi yang dibuat?	LunaCaffe POS adalah aplikasi desktop Java Swing untuk pemesanan kafe, manajemen pesanan, produk, akun pegawai, dan rekap transaksi.
Di mana letak OOP?	OOP terlihat pada model Aktor, Pegawai, Admin, Kasir, Pembeli, Menu, Pesanan, dan DetailPesanan. Class-class tersebut memiliki atribut, method, inheritance, abstraction, dan composition.
Contoh inheritance?	Admin dan Kasir extends Pegawai, sedangkan Pegawai extends Aktor.
Contoh abstraction?	Aktor adalah abstract class yang memiliki abstract method masukSistem().
Contoh polymorphism?	Pembeli dan Pegawai sama-sama override masukSistem(), tetapi perilakunya berbeda.
Contoh encapsulation?	Atribut harga, stok, password, dan totalHarga tidak diakses langsung dari luar, tetapi melalui method.
Mengapa memakai DAO?	Agar query SQL tidak bercampur dengan tampilan dan logika antarmuka.
Bagaimana stok aman?	Checkout memakai transaksi database dan query UPDATE stok dengan syarat stok >= qty. Jika gagal, rollback dilakukan.

BAB 10

Checklist Belajar Mandiri

Gunakan checklist ini sebelum presentasi UAS agar penguasaan materi lebih terarah.

- Saya bisa menjelaskan perbedaan class dan object menggunakan contoh Menu dan Pesanan.
- Saya bisa menggambar hierarki Aktor -> Pegawai -> Admin/Kasir dan Aktor -> Pembeli.
- Saya bisa menjelaskan mengapa password di Pegawai termasuk encapsulation.
- Saya bisa menjelaskan mengapa Pesanan memiliki List.
- Saya bisa menjelaskan fungsi DAO dan contoh method yang ada di MenuDAO atau OrderDAO.
- Saya bisa menjelaskan empat tabel SQLite dan relasinya.
- Saya bisa mendemokan login admin, tambah produk, checkout pelanggan, dan status pesanan selesai.
- Saya bisa menjelaskan bagaimana sistem mengurangi stok saat checkout.

Kesimpulan

LunaCaffe POS merupakan contoh penerapan PBO pada aplikasi bisnis sederhana. Project ini tidak hanya memakai class dan object, tetapi juga menunjukkan pewarisan, abstraksi, enkapsulasi, polymorphism, composition, MVC, DAO, dan database lokal. Dengan struktur tersebut, aplikasi lebih mudah dipahami sebagai bahan UAS dan lebih mudah dikembangkan ke tahap berikutnya.

Dokumen	Lokasi
Dokumentasi web	index.html
Demo visual	demo.html
Database lokal	lunacaffe.db
Aset gambar menu	src/main/resources/images/menu